DAW · BEZERO-INGURUNEKO PROGRAMAZIOA

Errepaso gida

JavaScript · DOM · BOM · Web APIak

Ikasleentzako errepaso dokumentua: kontzeptu nagusiak, kodea, ohiko akatsak, autotestak eta azterketa-plana, `edukiak/` eta `ariketak/` karpeten edukitik abiatuta.

2026ko azaroa

Bertsioa 1.0

Aurkibidea

1. Azkarreko orria (10 minutuko errepasoa)

2. Unitatea — Web arkitektura eta HTTP

3. Unitatea — JavaScript oinarriak

4. Unitatea — Arrayak, objektuak, prototipoak, klaseak eta moduluak

5. Unitatea — DOM, BOM eta Web APIak

6. Ohiko akatsen kontrol-zerrenda (azterketa aurretik)

7. Kode-erronkak (erantzunekin)

8. Azterketa prestatzeko plana

9. Glosarioa

10. Azken gogorarazpenak

1. Azkarreko orria (10 minutuko errepasoa)

Ideia	Zer jakin behar duzu
Internet vs. WWW	Internet sareen azpiegitura da (TCP/IP). WWW Interneten gaineko zerbitzua da (HTTP).
HTTP	Eskaera/erantzun protokoloa. GET datuak eskatzen ditu; POST datuak bidaltzen ditu.
Egoera-kodeak	2xx ondo, 3xx berbideraketa, 4xx bezeroaren errorea, 5xx zerbitzariaren errorea.
Aldagaiak	`const` lehenetsi; `let` balioa aldatuko bada; `var` ez erabili.
Konparazioa	`===` eta `!==` erabili beti (`==` motak bihurtzen ditu).
Funtzioak	Funtzioa balio bat da: aldagaian gorde, parametro gisa pasa, itzuli.
Arrayak	`map` → array berria; `filter` → baldintza betetzen dutenak; `reduce` → balio bakarra.
Objektuak	`{ gakoa: balioa }`; `obj.gakoa` edo `obj["gakoa"]`.
`this`	Nork deitu duen araberakoa da; metodoan objektua bera.
Prototipoak/Klaseak	JS prototipoetan oinarrituta dago; `class` sintaxi modernoa da.
DOM	HTMLaren zuhaitza: hautatu, sortu, aldatu, ezabatu.
BOM	Nabigatzailearen leihoa: `window`, `location`, `history`, `navigator`, timerrak.
Web API	Nabigatzailearen gaitasunak: `localStorage`, `geolocation`, `fetch`...

2. Unitatea — Web arkitektura eta HTTP

2.1 Internet eta WWW ez dira gauza bera

- **Internet**: elkarri konektatutako sareen sarea. Protokolo komuna: **TCP/IP**. Lehen konexioa: **ARPANET (1969)**.
- **WWW (World Wide Web)**: Interneten gainean funtzionatzen duen zerbitzua. Datuak **HTTP** bidez transmititzen dira. **Tim Berners-Leek** sortu zuen **CERNen, 1989an**.
- Beste Interneteko zerbitzuak: posta (SMTP), fitxategi-transferentzia (FTP), DNS…

2.2 TCP/IP modeloaren 4 geruzak

Geruza	Funtzioa	Adibideak
Sarbide edo fisikoa	Azpiegitura fisikoa, sare lokalak	Ethernet, Wi-Fi, kablea
Sarea	Helbideratzea eta bideratzea	IP
Garraioa	Datuak modu fidagarrian garraiatzea	TCP, UDP
Aplikazioa	Erabiltzailearen zerbitzuak	HTTP, HTTPS, DNS, SMTP

2.3 HTTP mezua

Eskaera (request):

```
GET /index.html HTTP/1.1
Host: www.adibidea.eus
User-Agent: Mozilla/5.0 ...
Accept: text/html
```

Erantzuna (response):

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1234

<!DOCTYPE html>
...
```

- **Eskaera**: metodoa + bidea + bertsioa → goiburuak → lerro hutsa → gorputza (aukerakoa).
- **Erantzuna**: egoera-lerroa → goiburuak → lerro hutsa → gorputza.

2.4 GET vs. POST

GET	POST
Datuak eskatzea	Datuak bidaltzea
Datuak URLean (`?gakoa=balioa`)	Datuak gorputzean
Luzeera mugatua, ikusgai	Luzeera handiagoa, ez ikusgai
Cache/lotarako egokia	Inprimakiak, datu sentikorrak
Ez da aldaketak egiteko erabili behar	Aldaketak egiteko erabil daiteke

2.5 Egoera-kodeak (ondo bereizi!)

Kodea	Esanahia
200	OK — eskaera ondo burutu da
201	Created — baliabidea sortu da
301/302	Mugitu da / aldi baterako mugitu da
304	Ez da aldatu (cache)
400	Bad Request — eskaera okerra
401	Unauthorized — autentikazioa falta da
403	Forbidden — baimenik ez
404	Not Found — baliabidea ez da aurkitu
500	Internal Server Error — zerbitzariaren errorea
502/503	Gateway okerra / zerbitzua ez dago erabilgarri

> ⚠️ Ohiko nahastea: **401 = nor zara?** / **403 = ez duzu baimenik** / **404 = ez dago**.

2.6 Nabigatzailearen arkitektura

Osagaia	Funtzioa
Erabiltzaile-interfazea	Tabs, barrak, menua, deskargak, konfigurazioa
Motorra (navigazioa)	URL karga, aurrera/atzera, JS alertak, hobespenak
Renderizatzailea	HTML/CSS interpretatu eta marraztu (Gecko, Blink, WebKit)
Komunikazioak	HTTP, FTP, MIME, cachea

Osagaia	Funtzioa
JS interpretea	Script kodea aztertu eta exekutatu
XML parserra	DOM zuhaitza memorian eraiki

2.7 Front-end / Back-end

- **Front-end**: erabiltzaileak ikusten duena → HTML + CSS + JavaScript. HTTP eskaerak bidal ditzake.
- **Back-end**: zerbitzariaren logika → eskaerak jaso eta erantzun.
- **DevTools** → **Network**: eskaerak, goiburuak, egoera-kodeak, denborak eta erantzunak aztertzeko.

✅ Autotest — 1. unitatea

1. Noiz sortu zen WWW eta nork?
2. Zer geruza da HTTP?
3. Zein da GET eta POSTen arteko desberdintasun nagusia?
4. 404 eta 500 kodea bereizi.
5. Zer da User-Agent?
6. Zein da renderizatzaile baten eginkizuna?

▼ Erantzunak

1. 1989an, Tim Berners-Leek, CERNen.
2. Aplikazio-geruza.
3. GETek datuak URLean bidaltzen ditu eta informazioa eskatzen du; POSTek datuak gorputzean bidaltzen ditu eta aldaketak egin ditzake.
4. 404: baliabidea ez da aurkitu (bezeroaren aldetik); 500: zerbitzariaren barne-errorea.
5. Nabigatzaileak zerbitzariari bidaltzen dion identifikazio-katea (izena, bertsioa, sistema eragilea...).
6. HTML/CSS interpretatu eta pantailan marraztea, elementuen tamaina eta posizioa kalkulatuz.

---

3. Unitatea — JavaScript oinarriak

3.1 Ezaugarriak

- Interpretatua (ez du konpilatu behar).
- **ECMAScript** estandarraren gainean eraikia.
- Objektuetara orientatua.
- **Ahulki eta dinamikoki tipatua**: aldagaiak ez du tipo finkorik.
- Bezero-inguruneko hizkuntza nagusia; zerbitzarian ere bai (Node.js).

3.2 Nola sartu HTMLan

```
<!-- Barne-kodea -->
<script>
  console.log("Kaixo");
</script>

<!-- Kanpoko fitxategia -->
<script src="app.js" defer></script>

<!-- Moduluak -->
<script type="module" src="app.js"></script>

<!-- Gertaeraren bidez -->
<button onclick="agurtu()">Kaixo</button>

<!-- JS desgaituta badago -->
<noscript>Gaitu JavaScript nabigatzailean.</noscript>
```

- `defer`: HTMLa osatu ondoren exekutatzen da, ordena mantenduz.
- `type="module"`: beti deferred, strict mode, scope propioa.

3.3 Irteerak eta sarrerak

Metodoa	Non	Oharra
`console.log()`	Kontsola	Debuggingerako
`alert()`	Leiho elkarrizketa	Blokeatzailea
`confirm()`	Leiho elkarrizketa	`true` / `false` itzultzen du
`prompt()`	Leiho elkarrizketa	String edo `null` (Utzi)
`element.innerHTML`	HTML	HTML kodea interpretatzen du
`element.textContent`	HTML	Testu hutsa
`document.write()`	Dokumentua	Saihestu, arriskutsua da

3.4 Aldagaiak eta scopea

Hitz gakoa	Scopea	Berrdefini daiteke?	Balioa aldatu?
`var`	Funtzioa	Bai	Bai
`let`	Blokea	Ez bloke berean	Bai
`const`	Blokea	Ez	Ez (baina objektuaren edukia alda daiteke)

```
const izena = "Ane";        // ezin da berriro esleitu
let adina = 18;             // alda daiteke
adina = 19;

{
  let barnekoa = 1;         // bloke honek soilik ikusten du
  var kanpokoa = 2;         // funtzio osoan ikusten da
}

const pertsona = { izena: "Ane" };
pertsona.izena = "Iker";    // ✅ objektuaren propietatea alda daiteke
// pertsona = {};           // ❌ const delako
```

- **Hoisting**: `var` altxatzen da (balioa `undefined`); `let` / `const` ere altxatzen dira baina **TDZ** (temporal dead zone) dute.
- Scope globala, funtzio-scopea eta bloke-scopea bereizi.

3.5 Tipoak

Tipoa	Adibidea	`typeof`
`number`	`42`, `3.14`, `NaN`, `Infinity`	`"number"`
`string`	`"kaixo"`, `'kaixo'`, `` `kaixo` ``	`"string"`
`boolean`	`true`, `false`	`"boolean"`
`object`	`{}`, `[]`, `new Date()`, `null`	`"object"`
`undefined`	aldagaia deklaratu, baliorik gabe	`"undefined"`
`function`	`function f(){}`	`"function"`

```
typeof null          // "object"  ⚠️ klasikoa
typeof undefined     // "undefined"
typeof []            // "object"
Array.isArray([])    // true
null === undefined   // false
null == undefined    // true
```

Balio falsy-ak: `false`, `0`, `-0`, `""`, `null`, `undefined`, `NaN`. Gainontzeko guztiak **truthy** dira (array hutsa eta objektu hutsa barne).

3.6 Bihurketa eta koakzioa

```
"5" + 2        // "52"   (concatenación)
"5" - 2        // 3      (bihurketa numerikoa)
Number("12")   // 12
Number("12px") // NaN
parseInt("12px")   // 12
parseFloat("3.5kg") // 3.5
String(123)    // "123"
Boolean(0)     // false
(0.1 + 0.2) === 0.3  // false ⚠️
```

- `+` eragilea da bereziena: string batekin batuta, dena string bihurtzen da.
- `prompt()` beti string itzultzen du (edo `null`); zenbakiak behar badituzu, `Number()` erabili.

3.7 Eragileak

```
// Aritmetikoak
+  -  *  /  %  **  ++  --

// Esleipena
=  +=  -=  *=  /=  %=

// Konparazioa
==   // balioa (mota bihurtzen du) ⚠️
===  // balioa eta mota ✅
!=   !==
>  <  >=  <=

// Logikoak
&&  ||  !

// Beste batzuk
condition ? balioa1 : balioa2   // ternarioa
typeof x
instanceof
```

3.8 Kontrol-egiturak

```
if (adina >= 18) {
  console.log("Nagusia");
} else if (adina >= 16) {
  console.log("Enantzintzakoa");
} else {
  console.log("Adingabea");
}

switch (eguna) {
  case "larunbata":
  case "igandea":
    console.log("Asteburua");
    break;
  default:
```

```
    console.log("Laneguna");
}
```

3.9 Begiztak

Egitura	Noiz erabili	Adibidea
`for`	Iterazio-kopurua ezaguna	`for (let i = 0; i < 5; i++)`
`while`	Baldintza bete arte	`while (x < 10)`
`do...while`	Gutxienez behin	`do { ... } while (x < 10)`
`for...in`	Objektuen **gakoak** (edo arrayen indizeak)	`for (const k in obj)`
`for...of`	Iteragarrien **balioak**	`for (const v of array)`
`forEach`	Arrayen elementuak	`array.forEach((v, i) => ...)`

```
const frutak = ["banana", "sagarra", "udarea"];

for (const fruta of frutak) {
  console.log(fruta);          // balioa
}

for (const indizea in frutak) {
  console.log(indizea, frutak[indizea]); // indizea
}

frutak.forEach((fruta, i) => console.log(i, fruta));
```

- `break`: begizta osoa uzten du.
- `continue`: iterazio hau saltatu eta hurrengora pasatzen da.
- ⚠️ Arrayekin ez erabili `for...in` ordena garrantzitsua bada.

3.10 Funtzioak

```
// Deklarazioa
function batu(a, b) {
  return a + b;
}

// Adierazpena
const kendu = function (a, b) {
  return a - b;
};

// Arrow function
const biderkatu = (a, b) => a * b;

// Parametro lehenetsia
function agurtu(izena = "ikasle") {
  return `Kaixo, ${izena}!`;
```

```
}

// Gainerako parametroak
function batuGuztiak(...zenbakiak) {
  return zenbakiak.reduce((acc, n) => acc + n, 0);
}
```

- `return` gabe funtzioak `undefined` itzultzen du.
- Arrow funtzioek **ez dute** `this` **propiorik** eta ezin dute `arguments` erabili.
- Funtzioa **callback** gisa pasa daiteke (`map`, `filter`, `addEventListener`...).

3.11 Funtzio erabilgarriak

String:

```
const s = "  Kaixo, Mundua!  ";
s.length                 // 18
s.trim()                 // "Kaixo, Mundua!"
s.toUpperCase()          // "  KAIXO, MUNDUA!  "
s.toLowerCase()
s.charAt(2)              // "K"
s.at(-1)                 // "!"  (negatiboak onartzen ditu)
s.indexOf("Mundua")      // 9
s.includes("Kaixo")      // true
s.slice(2, 7)            // "Kaixo"
s.substring(2, 7)        // "Kaixo"
s.split(", ")            // ["  Kaixo", "Mundua!  "]
s.replace("Mundua", "Denok")
s.repeat(2)
```

Zenbakiak / Math:

```
Number("42")             // 42
parseInt("42px")         // 42
parseFloat("3.14")       // 3.14
(3.14159).toFixed(2)     // "3.14" (string!)
Number.isNaN(NaN)        // true

Math.round(4.6)          // 5
Math.floor(4.9)          // 4
Math.ceil(4.1)           // 5
Math.abs(-7)             // 7
Math.max(3, 9, 2)        // 9
Math.min(3, 9, 2)        // 2
Math.pow(2, 8)           // 256
Math.random()            // [0, 1)

// Dadoa: 1etik 6ra
Math.floor(Math.random() * 6) + 1
```

⚠️ Ohiko akatsak — 2. unitatea

- `=` (esleipena) eta `==` / `===` (konparazioa) nahastea.

- `==` erabiltzea; beti `===`.
- `Math.floor(Math.random() * 6)` → 0 atera daiteke; `+ 1` behar da.
- `prompt()`-en emaitza zuzenean batzea (`"2" + 3` → `"23"`).
- `const` array/objektu batekin: edukia alda daitekeela ahaztea.
- Funtzioa parametrorik gabe deitzea eta `NaN` lortzea.
- `return` ahaztea.
- `let` / `const` scopea ez kontuan hartzea.

✅ Autotest — 2. unitatea

1. `let` eta `var`en arteko desberdintasun nagusia?
2. Noiz erabili `const` eta noiz `let`?
3. Zer da `NaN`? Nola egiaztatzen da?
4. `"5" + 2` eta `"5" - 2` emaitzak?
5. `==` eta `===` desberdintasuna.
6. `for...in` eta `for...of` noiz erabili?
7. Zer egiten du `continue`-k?
8. `Math.floor(Math.random() * 10) + 1` zer tartea ematen du?
9. Arrow funtzioen ezaugarri berezia?
10. Zer da callback bat?

> ▼ **Erantzunak**
>
> 1. `var` funtzio-scopea du eta berrdefini daiteke; `let` bloke-scopea du.
> 2. `const` balioa ez bada aldatuko; `let` bai.
> 3. Zenbaki ez-balioduna; `Number.isNaN(x)` edo `isNaN(x)`.
> 4. `"52"` eta `3`.
> 5. `==` motak bihurtzen ditu; `===` balioa eta mota konparatzen ditu.
> 6. `for...in` gakoetarako (objektuak); `for...of` balioetarako (arrayak, stringak).
> 7. Iterazio hau saltatu eta hurrengora joatea.
> 8. 1etik 10era.
> 9. Ez dute `this` propiorik; laburragoak dira.
> 10. Beste funtzio bati parametro gisa pasatzen zaion funtzioa.

---

4. Unitatea — Arrayak, objektuak, prototipoak, klaseak eta moduluak

4.1 Arrayak

```
const frutak = ["banana", "sagarra", "udarea"];
const zenbakiak = [4, 8, 15, 16, 23, 42];

frutak[0]           // "banana"
frutak.length       // 3
frutak.at(-1)       // "udarea"
```

Metodoa	Funtzioa	Aldatzen du?
`push(x)`	Amaieran gehitu	Bai
`pop()`	Azkena kendu eta itzuli	Bai
`unshift(x)`	Hasieran gehitu	Bai
`shift()`	Lehena kendu eta itzuli	Bai
`splice(i, n, ...)`	Kendu/gehitu posizioan	Bai
`slice(i, j)`	Zatia kopiatu	Ez
`concat(a)`	Arrayak batu	Ez
`join(sep)`	String bihurtu	Ez
`indexOf(x)`	Indizea aurkitu	Ez
`includes(x)`	Badagoen egiaztatu	Ez
`find(fn)`	Lehen elementua	Ez
`findIndex(fn)`	Lehen indizea	Ez
`sort()`	Ordenatu (string gisa!)	Bai
`toSorted()`	Ordenatu kopian	Ez
`reverse()`	Alderantziz jarri	Bai

```
// Zenbakizko ordena zuzena
zenbakiak.sort((a, b) => a - b);

// splice: 1 posiziotik 2 elementu kendu eta bi berri sartu
const berria = [...frutak];
berria.splice(1, 2, "kiwi", "mango");
```

4.2 Array funtzionalak (oso garrantzitsuak)

Metodoa	Itzultzen du	Adibidea
`forEach(fn)`	`undefined` (alboko efektua)	`arr.forEach(x => console.log(x))`
`map(fn)`	Array berria, elementu bakoitza eraldatuta	`[1,2,3].map(x => x*2)` → `[2,4,6]`
`filter(fn)`	Array berria, baldintza betetzen dutenak	`[1,2,3,4].filter(x => x%2===0)` → `[2,4]`
`reduce(fn, hasiera)`	Balio bakarra	`[1,2,3].reduce((a,b)=>a+b, 0)` → `6`
`every(fn)`	`true` denak betetzen badu	`[2,4].every(x => x%2===0)` → `true`
`some(fn)`	`true` batek betetzen badu	`[1,2].some(x => x>1)` → `true`

```
const prezioak = [100, 250, 400, 600];

// % 10, % 20 edo % 30eko deskontua
const deskontuak = prezioak.map((p) => {
  if (p <= 200) return p * 0.9;
  if (p <= 500) return p * 0.8;
  return p * 0.7;
});

// 300 baino garestiagoak
const garestiak = prezioak.filter((p) => p > 300);

// Batezbestekoa
const batezbestekoa = prezioak.reduce((acc, p) => acc + p, 0) / prezioak.length;

// 500etik hurbilen dagoen zenbakia
const hurbilena = [120, 480, 510, 450, 600].reduce((acc, n) =>
  Math.abs(500 - n) < Math.abs(500 - acc) ? n : acc
);
```

⚠️ `reduce`-n hasierako balioa (`0`, `[]`, `{}`) ematea gomendagarria da. ⚠️
`map` / `filter` / `reduce` ez dute jatorrizko arraya aldatzen; `forEach`-k ez du ezer itzultzen.

4.3 Objektuak

```
const pertsona = {
  izena: "Ane",
  adina: 21,
  helbidea: { hiria: "Usurbil", kalea: "Nagusia" },
  agurtu() {
    return `Kaixo, ${this.izena}`;
  },
};

pertsona.izena            // "Ane"
```

```
pertsona["adina"]          // 21
pertsona.helbidea.hiria    // "Usurbil"
pertsona.agurtu()          // "Kaixo, Ane"

pertsona.email = "a@b.eus";   // gehitu
delete pertsona.adina;        // ezabatu

Object.keys(pertsona);        // gakoak
Object.values(pertsona);      // balioak
Object.entries(pertsona);     // [gakoa, balioa] bikoteak
```

- Objektuak **erreferentziaz** pasatzen dira: `const b = a` eginez gero, bi aldagaiek objektu bera seinalatzen dute.
- Objektuen arrayak oso ohikoak dira:

```
const ikasleak = [
  { izena: "Ane", nota: 8 },
  { izena: "Iker", nota: 6 },
];

const gaindituak = ikasleak.filter((i) => i.nota >= 5);
const notak = ikasleak.map((i) => i.nota);
ikasleak.sort((a, b) => b.nota - a.nota);
```

4.4 Prototipoak

JavaScripten herentzia **prototipo-katearen** bidez egiten da.

```
function Pertsona(izena, adina) {
  this.izena = izena;
  this.adina = adina;
}

Pertsona.prototype.agurtu = function () {
  return `Kaixo, ${this.izena}`;
};

const ane = new Pertsona("Ane", 21);
ane.agurtu();                 // "Kaixo, Ane"
ane instanceof Pertsona;      // true
```

- `new` gabe deituta, `this` ez da objektu berria → errorea edo balio okerrak.
- Metodoak **prototipoan** jartzea eraginkorra da: instantzia guztiek partekatzen dituzte.
- Propietateak instantzian egon ohi dira; metodoak prototipoan.

4.5 Klaseak (sintaxi modernoa)

```
class Pertsona {
  constructor(izena, adina) {
    this.izena = izena;
    this.adina = adina;
  }
```

```
  agurtu() {
    return `Kaixo, ${this.izena}`;
  }

  get izena() {
    return this._izena;
  }

  set izena(balioa) {
    if (!balioa) throw new Error("Izena ezin da hutsik egon");
    this._izena = balioa;
  }

  static espeziea() {
    return "Gizakia";
  }
}

const ane = new Pertsona("Ane", 21);
ane.agurtu();
Pertsona.espeziea();     // ✅ klasearen bidez
// ane.espeziea();        // ❌ instantziatik ezin da
```

Herentzia:

```
class Ikaslea extends Pertsona {
  constructor(izena, adina, ikastetxea) {
    super(izena, adina);          // derrigorrezkoa
    this.ikastetxea = ikastetxea;
  }

  agurtu() {
    return `${super.agurtu()} (${this.ikastetxea})`;
  }
}
```

Prototipoak	Klaseak
`function` konstruktorea	`class` sintaxia
`Ctor.prototype.metodoa = ...`	Metodoa klasearen barruan
`Object.create(...)`	`extends` / `super()`
Zaharragoa, malguagoa	Argia eta irakurgarria

> Klaseak ez dira objektuak, **txantiloiak** baizik. `static` metodoak klaseari dagozkio, ez instantziei.

4.6 Getterrak eta setterrak

- `get` / `set` propietate gisa erabiltzen dira (`obj.adina`, ez `obj.adina()`).
- Erabilera: **enkapsulazioa**, **balidazioa** eta **kontrola**.

```
class Erabiltzailea {
  constructor(nick, pasahitza) {
    this._nick = nick;
    this._pasahitza = pasahitza;
  }

  get nick() { return this._nick; }

  set nick(balioa) {
    if (typeof balioa !== "string" || /\s/.test(balioa)) {
      throw new Error("Nick-ak ezin du zuriunerik izan");
    }
    this._nick = balioa;
  }
}
```

4.7 Moduluak

`mates.js`:

```
export const PI = 3.1416;

export function batu(a, b) {
  return a + b;
}

export default class Kalkulagailua {
  biderkatu(a, b) {
    return a * b;
  }
}
```

`index.html`:

```
<script type="module">
  import Kalkulagailua, { PI, batu } from "./mates.js";
  // import * as mates from "./mates.js";

  console.log(PI, batu(2, 3), new Kalkulagailua().biderkatu(2, 3));
</script>
```

- **Named export**: bat baino gehiago, izen berekoak inportatzean.
- **Default export**: modulu bakoitzeko bakarra, edozein izenekin inporta daiteke.
- `<script type="module">` behar da; `file://` bidez arazoak izan daitezke → **zerbitzari lokala** erabili.
- `import` bideak zuzenak izan behar dira (`./`, `../`, luzapena).

⚠️ Ohiko akatsak — 3. unitatea

- `sort()` zenbakietan: `[10, 2, 1].sort()` → `[1, 10, 2]` (string gisa ordenatzen du).
- `map` eta `forEach` nahastea: `map`-ek array berria itzultzen du.

- `filter` eta `find` nahastea: `find` -ek elementua; `filter` -ek arraya.
- `slice` eta `splice` nahastea: `splice` -k aldatzen du.
- `reduce` -n hasierako balioa ahaztea.
- Objektua `const` bada ere, propietateak alda daitezkeela ez jakitea.
- `this` galtzea callback arruntean.
- Klasean `super()` deitu aurretik `this` erabiltzea.
- `export default` bat baino gehiago jartzea.
- `type="module"` ahaztea.

✅ Autotest — 3. unitatea

1. `push` eta `unshift` desberdintasuna?
2. `slice` eta `splice` desberdintasuna?
3. Zer itzultzen du `map` -ek? Eta `forEach` -ek?
4. Nola lortzen da array baten batura `reduce` -rekin?
5. Nola ordenatzen dira zenbakiak array batean?
6. Zer da `this` metodo baten barruan?
7. Noiz erabili `get` / `set` ?
8. Nola heredatzen da klase bat?
9. Zer da `static` metodo bat?
10. Nola inportatzen da `export default` bat?

> ▼ **Erantzunak**
>
> 1. `push` amaieran gehitzen du; `unshift` hasieran.
> 2. `slice` kopiatzen du (ez du aldatzen); `splice` aldatzen du.
> 3. `map` array berria; `forEach` `undefined` .
> 4. `arr.reduce((acc, x) => acc + x, 0)` .
> 5. `arr.sort((a, b) => a - b)` .
> 6. Metodoa deitu duen objektua.
> 7. Balidazioa, enkapsulazioa eta propietateen kontrola egiteko.
> 8. `class Filia extends Gurasoa` eta `super(...)` .
> 9. Klasearen beraren gainean dagoena, instantziatik deitu ezin dena.
> 10. `import EdozeinIzen from "./modulua.js"` .

---

5. Unitatea — DOM, BOM eta Web APIak

5.1 DOM zer da?

- **Document Object Model**: HTML dokumentuaren zuhaitz-egitura.
- Nabigatzaileak `document` objektua sortzen du kargatzean.
- Nodo motak: **elementua**, **atributua** eta **testua**.
- DOM manipulatzen badugu, erabiltzaileak beste dokumentu bat ikus dezake.

```
document
└── html
    ├── head
    │   └── title
    └── body
        ├── h1
        └── p
```

5.2 Elementuak hautatzea

Metodoa	Itzultzen du	Adibidea
`getElementById("id")`	Elementu bakarra	`document.getElementById("mezua")`
`querySelector("css")`	Lehen elementua	`document.querySelector(".karta")`
`querySelectorAll("css")`	NodeList (guztiak)	`document.querySelectorAll("p")`
`getElementsByClassName("c")`	HTMLCollection (bizia)	`document.getElementsByClassName("nabarmendu")`
`getElementsByTagName("p")`	HTMLCollection (bizia)	`document.getElementsByTagName("a")`
`getElementsByName("n")`	NodeList	`document.getElementsByName("izena")`

```js
const parrafoak = document.querySelectorAll("p");
console.log(parrafoak.length);

parrafoak[0];                 // lehena
parrafoak[2].textContent;     // hirugarrena

// NodeList bat array bihurtu
[...parrafoak].map((p) => p.textContent);

// Elementu baten barruan bilatu
const karta = document.querySelector(".karta");
const estekak = karta.querySelectorAll("a");
```

5.3 Edukia eta atributuak aldatzea

```
const elementua = document.getElementById("mezua");

elementua.textContent = "Testu hutsa";            // segurua
elementua.innerHTML = "<strong>Testua</strong>";  // HTMLa interpretatzen du

elementua.style.color = "red";
elementua.classList.add("aktiboa");
elementua.classList.remove("ezkutua");
elementua.classList.toggle("nabarmendua");
elementua.classList.contains("aktiboa");          // true/false

elementua.setAttribute("data-egoera", "ok");
elementua.getAttribute("data-egoera");
elementua.removeAttribute("data-egoera");
elementua.dataset.egoera;                          // "ok"

const irudia = document.querySelector("img");
irudia.src = "argazkia.jpg";
irudia.alt = "Deskribapena";
```

⚠️ Erabiltzailearen datuak `innerHTML`-n sartzeak **XSS** arriskua du. Testurako `textContent` erabili.

5.4 Elementuak sortu, gehitu eta ezabatu

```
const lista = document.getElementById("lista");

// Sortu eta gehitu
const li = document.createElement("li");
li.textContent = "Elementu berria";
lista.appendChild(li);

// Testu-nodoa
const testua = document.createTextNode("Kaixo");
li.appendChild(testua);

// Posizio zehatzean
const berria = document.createElement("li");
berria.textContent = "Lehenengoa";
lista.insertBefore(berria, lista.firstElementChild);

// Elementu baten aurretik/atzetik
elementua.before(berria);
elementua.after(berria);

// Ordeztu eta ezabatu
lista.replaceChild(berria, lista.children[1]);
elementua.remove();
lista.removeChild(lista.lastElementChild);

// Klonatu
const kopia = li.cloneNode(true);   // true = ondorengoekin
```

5.5 Nabigazioa

Propietatea	Esanahia
`parentElement`	Guraso-elementua
`children`	Seme-elementuak (HTMLCollection)
`childNodes`	Nodo guztiak (testu-zuriuneak barne!)
`firstElementChild` / `lastElementChild`	Lehen/azken seme-elementua
`nextElementSibling` / `previousElementSibling`	Anai-arrebak
`contains(nodo)`	Barruan dagoen egiaztatu

> ⚠️ `childNodes` eta `children` ez dira berdinak: lehenak testu-nodoak eta iruzkinak sartzen ditu.

5.6 Gertaerak (events)

```
const botoia = document.getElementById("botoia");

botoia.addEventListener("click", (event) => {
  event.preventDefault();       // portaera lehenetsia ekidin
  console.log("Klik!", event.target);
});

// Inprimakiak
const forma = document.querySelector("form");
forma.addEventListener("submit", (e) => {
  e.preventDefault();
  const izena = document.querySelector("#izena").value.trim();
  if (!izena) {
    alert("Idatzi izena");
    return;
  }
  // ...
});

// DOM prest dagoenean
document.addEventListener("DOMContentLoaded", () => {
  console.log("DOM prest");
});

// Teklatua
document.addEventListener("keydown", (e) => {
  if (e.key === "Enter") console.log("Enter");
});
```

- `DOMContentLoaded`: HTMLa parseatu da, baina irudiak ez.
- `load`: dena kargatu da (irudiak barne).
- Gertaerak **burbuileatzen** dira; `stopPropagation()` erabili behar denean.

5.7 BOM — Browser Object Model

`window` da nabigatzailearen leihoaren objektu globala.

Objektua	Erabilera
`window.innerWidth/innerHeight`	Ikusgai den eremuaren tamaina
`window.outerWidth/outerHeight`	Leiho osoaren tamaina
`window.screenX/screenY`	Leihoaren posizioa pantailan
`screen.width/height/availWidth/availHeight`	Pantailaren informazioa
`window.open(url, izena, ezaugarriak)`	Leiho berria ireki
`window.close()`	Leihoa itxi
`window.moveTo/resizeTo`	Mugitu/redimensionatu (murriztua)
`location.href`	URL osoa
`location.pathname`, `search`, `hash`, `host`, `protocol`	URLaren zatiak
`location.assign(url)` / `replace(url)` / `reload()`	Nabigatu/berritu
`history.length`, `back()`, `forward()`, `go(n)`	Historiala
`navigator.userAgent`, `language`, `onLine`, `geolocation`	Nabigatzailearen informazioa
`alert`, `confirm`, `prompt`	Erabiltzailearekin komunikatu
`setTimeout(fn, ms)`	Behin exekutatu
`setInterval(fn, ms)`	Aldiro exekutatu

```
// Timerrak
const id = setInterval(() => {
  console.log(new Date().toLocaleTimeString());
}, 1000);

setTimeout(() => clearInterval(id), 5000);

// Leiho berria
const leihoa = window.open("", "_blank", "width=400,height=300");
if (!leihoa) {
  alert("Popup blokeatzailea aktibo dago");
}

// URL parametroak
const params = new URLSearchParams(location.search);
console.log(params.get("bilatu"));
```

5.8 Web APIak

APIa	Funtzioa
`localStorage` / `sessionStorage`	Datu txikiak gorde (stringak)
`Geolocation`	Kokapena lortu
`fetch`	HTTP eskaerak (async)
`Notification`	Jakinarazpenak
`Canvas` / `WebGL`	Marrazketa
`Clipboard`	Arbelera kopiatu

Storage:

```
const datua = { izena: "Ane", adina: 21 };

localStorage.setItem("erabiltzailea", JSON.stringify(datua));

const gordea = localStorage.getItem("erabiltzailea");
const objektua = gordea ? JSON.parse(gordea) : null;

localStorage.removeItem("erabiltzailea");
localStorage.clear();

// localStorage: iraunkorra, 5-10 MB inguru
// sessionStorage: fitxa/tab bakoitzeko, ixtean ezabatzen da
```

Geolocation:

```
if (!navigator.geolocation) {
  console.error("Geolocation ez da onartzen");
} else {
  navigator.geolocation.getCurrentPosition(
    (posizioa) => {
      const { latitude, longitude, accuracy } = posizioa.coords;
      console.log(latitude, longitude, accuracy);
    },
    (errorea) => {
      console.error("Errorea:", errorea.message);
    }
  );
}
```

- HTTPS edo `localhost` behar da.
- Baimena eskatzen du; erantzuna **asinkronoa** da.

Fetch:

```
async function datuakKargatu() {
  try {
    const erantzuna = await fetch("datuak.json");
```

```
    if (!erantzuna.ok) throw new Error(`HTTP ${erantzuna.status}`);
    const datuak = await erantzuna.json();
    console.log(datuak);
  } catch (errorea) {
    console.error("Errorea:", errorea.message);
  }
}
```

⚠️ Ohiko akatsak — 4. unitatea

- Script-a DOMa sortu aurretik exekutatzea → `null` lortu.
- `getElementById` eta `querySelector("#id")` nahastea.
- `querySelectorAll`-ek arraya ez den NodeList bat itzultzen duela ahaztea.
- `textContent` behar den tokian `innerHTML` erabiltzea.
- `children` vs. `childNodes` nahastea.
- Inprimakian `e.preventDefault()` ahaztea → orria berriz kargatzen da.
- `localStorage`-n objektuak zuzenean gorde nahi izatea (lehen `JSON.stringify`).
- `setInterval` garbitzea ahaztea → memoria/isuria.
- Popupak: `window.open` blokeatuta egon daiteke.
- Moduluak `file://` bidez irekitzea → CORS errorea.

✅ Autotest — 4. unitatea

1. Zer da DOM? Eta BOM?
2. `getElementById` eta `querySelector` desberdintasuna?
3. `textContent` eta `innerHTML` desberdintasuna?
4. Nola sortzen da elementu berri bat eta gehitzen zaio gurasoari?
5. `children` eta `childNodes` desberdintasuna?
6. Zer gertatzen da `addEventListener("submit")`-n `preventDefault()` ez bada deitzen?
7. `DOMContentLoaded` eta `load` desberdintasuna?
8. Non gordetzen da `localStorage`? Zer gorde dezake?
9. Zer behar da Geolocation erabiltzeko?
10. `setTimeout` eta `setInterval` desberdintasuna?

▼ **Erantzunak**

1. DOM: HTML dokumentuaren zuhaitza; BOM: nabigatzailearen leihoaren objektu-eredua.
2. `getElementById` ID bidez elementu bakarra; `querySelector` CSS hautatzaile bidez lehena.
3. `textContent` testu hutsa; `innerHTML` HTML kodea interpretatzen du.
4. `document.createElement("li")`, edukia ezarri eta `gurasoa.appendChild(li)`.
5. `children` elementuak soilik; `childNodes` testu-nodoak eta iruzkinak ere bai.
6. Inprimakia bere portaera lehenetsian bidaltzen da eta orria berriz kargatzen da.
7. `DOMContentLoaded` HTMLa parseatu denean; `load` baliabide guztiak kargatu direnean.
8. Nabigatzailean, string gisa, iraunkorra (5-10 MB inguru).
9. HTTPS/localhost, erabiltzailearen baimena eta `navigator.geolocation`.
10. `setTimeout` behin exekutatzen da; `setInterval` aldiro.

---

6. Ohiko akatsen kontrol-zerrenda (azterketa aurretik)

- ☐ `=` eta `===` nahasten ez ditut.
- ☐ `const` eta `let` ondo aukeratzen ditut.
- ☐ `prompt`-en emaitza zenbakira bihurtzen dut.
- ☐ `Math.random()` tarte zuzenarekin erabiltzen dut.
- ☐ Funtzioek `return` behar dutenean jartzen dut.
- ☐ Arrow funtzioetan `this` ez dudala erabiltzen gogoratzen dut.
- ☐ `map` / `filter` / `reduce` / `forEach` desberdintzen ditut.
- ☐ `sort`-i konparatzailea pasatzen diot zenbakietan.
- ☐ `slice` eta `splice` bereizten ditut.
- ☐ Objektuak erreferentziaz pasatzen direla badakit.
- ☐ `this` metodoaren deitzailearen araberakoa dela badakit.
- ☐ Klasean `super()` lehenengo deitzen dut.
- ☐ `static` metodoak klasetik deitzen ditut.
- ☐ `export default` bakarra dela badakit.
- ☐ `<script type="module">` erabiltzen dut moduluetan.
- ☐ DOMa prest dagoenean exekutatzen dut kodea.
- ☐ `querySelectorAll` ez dela array bat gogoratzen dut.
- ☐ `textContent` eta `innerHTML` bereizten ditut.
- ☐ `children` eta `childNodes` bereizten ditut.
- ☐ Inprimakietan `preventDefault()` deitzen dut.
- ☐ `localStorage`-ra `JSON.stringify` erabiliz gordetzen dut.
- ☐ `JSON.parse` erabiltzen dut irakurtzean.
- ☐ Timerrak `clearTimeout` / `clearInterval` bidez garbitzen ditut.
- ☐ `window.open` blokea daitekeela badakit.
- ☐ Geolocation asinkronoa eta baimenaren mende dagoela badakit.
- ☐ `fetch`-en `response.ok` egiaztatzen dut.
- ☐ Moduluak zerbitzari lokaletik probatzen ditut.
- ☐ Kontsolako erroreak irakurtzen ditut, ez ditut isildu.
- ☐ Aldagai-izen deskribatzaileak erabiltzen ditut.

---

7. Kode-erronkak (erantzunekin)

7.1 FizzBuzz

Idatzi 1etik 100era: 3ren multiploak `"Fizz"`, 5en multiploak `"Buzz"`, bien multiploak `"FizzBuzz"`.

▼ **Soluzioa**

```
for (let i = 1; i <= 100; i++) {
  if (i % 15 === 0) console.log("FizzBuzz");
  else if (i % 3 === 0) console.log("Fizz");
  else if (i % 5 === 0) console.log("Buzz");
  else console.log(i);
}
```

7.2 Arrayen estatistikak

Array batetik bikoitiak, batura eta batezbestekoa lortu.

▼ **Soluzioa**

```
const zenbakiak = [4, 7, 10, 13, 22, 5];

const bikoitiak = zenbakiak.filter((n) => n % 2 === 0);
const batura = zenbakiak.reduce((acc, n) => acc + n, 0);
const batezbestekoa = batura / zenbakiak.length;

console.log(bikoitiak, batura, batezbestekoa);
```

7.3 Objektuen arraya

Ikasleen array batetik gaindituak lortu, notaren arabera ordenatu eta izenak bistaratu.

▼ **Soluzioa**

```
const ikasleak = [
  { izena: "Ane", nota: 8 },
  { izena: "Iker", nota: 4 },
  { izena: "Maite", nota: 9 },
];

const gaindituak = ikasleak
  .filter((i) => i.nota >= 5)
  .sort((a, b) => b.nota - a.nota)
  .map((i) => `${i.izena}: ${i.nota}`);

console.log(gaindituak);
```

7.4 Klasea eta balidazioa

Sortu `Kontua` klasea: `jabea`, `saldoa` (hasieran 0), `sartu(kopurua)`, `atera(kopurua)` (saldoa nahikoa ez bada, errorea).

▼ **Soluzioa**

```
class Kontua {
  constructor(jabea) {
    this.jabea = jabea;
    this.saldoa = 0;
  }

  sartu(kopurua) {
    if (kopurua <= 0) throw new Error("Kopurua positiboa izan behar da");
    this.saldoa += kopurua;
  }

  atera(kopurua) {
    if (kopurua > this.saldoa) throw new Error("Saldoa nahikoa ez");
    this.saldoa -= kopurua;
  }
}

const kontua = new Kontua("Ane");
kontua.sartu(100);
kontua.atera(30);
console.log(kontua.saldoa); // 70
```

7.5 DOM: gonbidatuen zerrenda

Inprimaki bat: izena idatzi, botoia sakatu, zerrendara gehitu eta kontagailua eguneratu. Izen hutsa ez onartu.

▼ **Soluzioa**

```
<form id="forma">
  <input id="izena" type="text" placeholder="Gonbidatuaren izena" />
  <button type="submit">Gehitu</button>
</form>
<ul id="lista"></ul>
<p>Guztira: <span id="kopurua">0</span></p>

<script>
  const forma = document.getElementById("forma");
  const izena = document.getElementById("izena");
  const lista = document.getElementById("lista");
  const kopurua = document.getElementById("kopurua");

  forma.addEventListener("submit", (e) => {
    e.preventDefault();
    const balioa = izena.value.trim();
    if (!balioa) {
      alert("Idatzi izen bat");
      return;
    }

    const li = document.createElement("li");
    li.textContent = balioa;
    lista.appendChild(li);

    kopurua.textContent = lista.children.length;
    izena.value = "";
    izena.focus();
  });
</script>
```

7.6 Storage: azken bisita

Orria kargatzean, `localStorage`-tik izena irakurri eta agurtu; botoi batekin izena gorde.

▼ **Soluzioa**

```
<input id="izena" />
<button id="gorde">Gorde</button>
<p id="mezua"></p>

<script>
  const mezua = document.getElementById("mezua");
  const izena = document.getElementById("izena");

  function erakutsi() {
    const gordea = localStorage.getItem("izena");
    mezua.textContent = gordea ? `Kaixo, ${gordea}!` : "Kaixo, ezezagun!";
  }

  document.getElementById("gorde").addEventListener("click", () => {
    const balioa = izena.value.trim();
    if (balioa) localStorage.setItem("izena", balioa);
    erakutsi();
  });

  erakutsi();
</script>
```

---

8. Azterketa prestatzeko plana

Saioa	Edukia	Denbora
1	HTTP, arkitektura, egoera-kodeak	45 min
2	Aldagaiak, tipoak, eragileak, kontrol-egiturak	60 min
3	Funtzioak, string/Math, begiztak	60 min
4	Arrayak eta array funtzionalak	60 min
5	Objektuak, prototipoak, klaseak, moduluak	90 min
6	DOM: hautatu, aldatu, sortu, gertaerak	90 min
7	BOM, storage, geolocation, fetch	60 min
8	Autotest osoa + kode-erronkak	90 min

Ebaluazio-irizpide posibleak

Atala	Pisua	Zer baloratzen da
Kontzeptuak	%30	Terminologia, HTTP, DOM/BOM, OOP
Kodea	%40	Sintaxia, logika, funtzioen/arrayen erabilera
Arazketa	%20	Erroreak ulertu eta zuzentzea
Estiloa	%10	Izenak, indentazioa, iruzkinak, egitura

9. Glosarioa

Euskara	Gaztelania	Ingelera
Aldagaia	Variable	Variable
Balioa	Valor	Value
Funtzioa	Función	Function
Parametroa	Parámetro	Parameter
Itzultzeko balioa	Valor de retorno	Return value
Arraya	Array / Arreglo	Array
Objektua	Objeto	Object
Propietatea	Propiedad	Property
Metodoa	Método	Method
Prototipoa	Prototipo	Prototype
Klasea	Clase	Class
Herentzia	Herencia	Inheritance
Modulua	Módulo	Module
Gertaera	Evento	Event
Hautatzailea	Selector	Selector
Nodoa	Nodo	Node
Zuhaitza	Árbol	Tree
Biltegiratzea	Almacenamiento	Storage
Eskaera	Petición	Request
Erantzuna	Respuesta	Response
Egoera-kodea	Código de estado	Status code
Goiburua	Cabecera	Header
Zerbitzaria	Servidor	Server
Bezeroa	Cliente	Client

10. Azken gogorarazpenak

1. **Ulertu, ez memorizatu soilik**: kodea eskuz idatzi eta probatu.
2. **Kontsola irakurri**: errore-mezuek pista zuzena ematen dute.
3. **Kasuak probatu**: array hutsa, zero, `null`, testu hutsa, zenbaki negatiboak.
4. **Funtzio txikiak**: zati txikitan banatu eta bakoitza probatu.
5. **Izen onak**: `x`, `d`, `a` erabili beharrean, `erabiltzailea`, `data`, `arraya`.
6. **Iruzkinak**: zergatik egiten duzun azaldu, zer egiten duen soilik ez.
7. **Bertsio-kontrola**: aldaketak maiz gorde; akatsak atzera botatzeko.

> **Azterketan**: irakurri enuntziatua bi aldiz, idatzi kodea zatika, probatu kasu arrunt batekin eta ertz-kasu batekin, eta ez ahaztu `return`, `preventDefault()` eta `JSON.stringify/parse`.

Dokumentu hau `edukiak/` (teoria) eta `ariketak/` (praktika) karpeten edukitik abiatuta prestatu da, ikasleen errepasorako.